

## **SIMATS ENGINEERING**

### **CO4 - ASSESSMENT TOOL 2**

#### **Industry Problem Solving Task**

**COURSE NAME: Deep Learning**

**COURSE CODE: MLA0408**

#### **COURSE OUTCOME COVERED**

**CO 1:** Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

**Assessment Tool Weightage: 40%**

---

#### **QUESTIONS:**

**Total Marks:25**

##### **Q1. Transfer Learning**

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

##### **Q2. DenseNet and PixelNet**

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

##### **Q3. Bidirectional RNN and Encoder–Decoder**

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

##### **Q4. BPTT and LSTM**

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

##### **Q5. Integrated Deep Learning Solution**

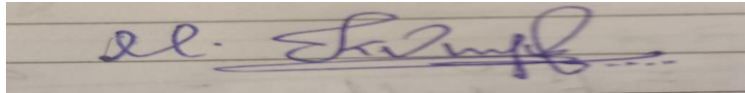
A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for

caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

**Code of Conduct:**

I KUMARAN M(192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



**Signature of the student:**

**RUBRICS FOR CALCULATION:**

| Criteria                            | Weightage | Level 4 – Exemplary                                                          | Level 3 – Proficient                                 | Level 2 – Developing                           | Level 1 – Beginning                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility        | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools           | Minimal or incorrect use of tools                 |
| Analysis                            | 10%       | Thorough analysis with validated results and performance evaluation          | Good analysis with reasonable validation             | Basic analysis with limited validation         | Poor or no analysis                               |

# ANSWERS

## Q1. Transfer Learning for Skin Cancer Detection

### Problem:

A healthcare startup wants to detect skin cancer from dermoscopic images, but only a limited number of labeled images are available. Training a deep CNN from scratch would require a very large dataset. Therefore, transfer learning using a pre-trained CNN is suitable.

Proposed Model: EfficientNet-B0

I would select EfficientNet-B0 as the pre-trained CNN because it provides a good balance between accuracy, computational cost, and model size. It can use features learned from large datasets such as ImageNet and adapt them to skin-lesion classification.

### Architecture:

Input Dermoscopic Image → Pre-processing → Pre-trained EfficientNet → Feature Extraction → Fine-tuned Layers → Fully Connected Layer → Skin Cancer Prediction

### Step 1: Image Pre-processing

- Resize images to the input size required by the network.
- Normalize pixel values.
- Apply augmentation such as rotation, horizontal/vertical flipping, cropping and slight brightness changes.
- Augmentation helps increase the effective size of the limited dataset.

### Step 2: Feature Extraction

Initially, the convolutional layers of EfficientNet are frozen.

The pre-trained network already contains useful visual features such as:

- Edges
- Shapes
- Textures

- Patterns
- Lesion structures

A new classification layer is added on top.

### **Step 3: Fine-Tuning**

After training the new classification layer, some of the deeper layers are unfrozen. The model is then trained with a small learning rate so that the learned features can adapt to dermoscopic images.

### **A typical arrangement is:**

EfficientNet-B0 backbone → Global Average Pooling → Dropout → Dense Layer → Softmax/Sigmoid

### **For binary classification:**

Output = Benign / Malignant

### **Why Freeze Some Layers?**

The early CNN layers learn general features such as edges and textures. These features are useful for many image-recognition tasks, so freezing them:

- Reduces training time.
- Reduces computational requirements.
- Prevents overfitting.
- Works well with a small dataset.

### **Why Fine-Tune Deeper Layers?**

The deeper layers contain more task-specific visual representations. Skin lesions have characteristics such as:

- Asymmetry
- Color variation
- Irregular borders
- Texture patterns
- Lesion structures

Fine-tuning allows the network to learn these domain-specific characteristics.

**Advantages:**

1. Requires less training data.
2. Reduces training time.
3. Provides better feature extraction.
4. Reduces overfitting on a small dataset.
5. Improves classification performance.
6. Requires less computational power than training from scratch.

**Conclusion:**

Transfer learning with EfficientNet-B0 is suitable for skin cancer detection because the model can reuse general image features learned from a large dataset. Freezing early layers and fine-tuning deeper layers allows the model to learn skin-lesion-specific characteristics while reducing overfitting. Therefore, transfer learning can provide better classification performance with limited labeled medical images.

## **Q2. DenseNet and PixelNet for Smart City Road-Scene Analysis**

### **Problem:**

A smart city requires an automated system that identifies:

- Roads
- Pedestrians
- Vehicles
- Traffic signs
- Other objects

at the pixel level. Therefore, the system requires both strong feature extraction and accurate semantic segmentation.

### **Proposed Architecture:**

The system can use DenseNet as the feature-extraction backbone and PixelNet-style pixel-level prediction for segmentation.

### **Architecture:**

Input Road Image



Pre-processing



DenseNet Feature Extraction



Dense Feature Maps



Pixel-level Feature Processing



Pixel Classification / Segmentation



Output Segmentation Map

## 1. DenseNet

DenseNet uses dense connections between layers.

Instead of passing information only from one layer to the next, each layer receives feature maps from previous layers.

### Conceptually:

Layer 1  $\rightarrow$  Layer 2  $\rightarrow$  Layer 3  $\rightarrow$  Layer 4

with dense connections such as:

Layer 1  $\rightarrow$  Layer 3

Layer 2  $\rightarrow$  Layer 4

This improves information and gradient flow.

## 2. Feature Reuse

DenseNet reuses previously learned features instead of learning the same features repeatedly.

This provides:

- Better feature propagation.
- Better gradient flow.
- Efficient parameter usage.
- Strong representation of fine details.

## 3. Pixel-Level Segmentation

PixelNet-style processing produces a classification for individual pixels.

### For example:

| Pixel Class  | Meaning             |
|--------------|---------------------|
| Road         | Road surface        |
| Vehicle      | Cars, buses, trucks |
| Pedestrian   | People              |
| Traffic Sign | Road signs          |
| Background   | Other regions       |

Every pixel is assigned to an appropriate class.

## **4. Improved Feature Reuse**

DenseNet passes low-level and high-level information throughout the network.

Low-level features help identify:

- Edges
- Boundaries
- Small objects

High-level features help identify:

- Vehicles
- Pedestrians
- Signs
- Road regions

Combining both improves segmentation quality.

## **5. Improved Segmentation Accuracy**

Pixel-level prediction is more accurate than simply predicting a bounding box because it identifies the exact region occupied by an object.

For example, instead of saying:

Vehicle → bounding box

the system can determine:

Vehicle → exact pixels belonging to the vehicle.

This is useful for autonomous traffic analysis.

## **6. Computational Efficiency**

DenseNet encourages feature reuse and can achieve strong performance with relatively efficient parameter usage.

The system can further improve efficiency using:

- Smaller input resolutions where appropriate.
- Batch processing.
- GPU acceleration.
- Model pruning or quantization for deployment.

## **Applications:**

The proposed system can be used for:

- Traffic monitoring.
- Pedestrian detection.
- Road-condition analysis.

- Traffic sign recognition.
- Vehicle counting.
- Smart traffic management.
- Autonomous driving assistance.

**Conclusion:**

Combining DenseNet for feature extraction with pixel-level segmentation provides strong feature reuse and accurate road-scene understanding. Dense connections improve information propagation, while pixel-level classification provides detailed segmentation. Therefore, the architecture is suitable for intelligent smart-city traffic monitoring.

### **Q3. Bidirectional RNN and Encoder–Decoder for Multilingual Translation**

#### **Problem:**

A multinational customer service company needs a multilingual chatbot that translates customer queries while preserving their contextual meaning.

A suitable solution is an Encoder–Decoder architecture using Bidirectional RNNs.

#### **Proposed Architecture:**

Input Sentence



Embedding Layer



Bidirectional RNN Encoder



Context Representation



Decoder RNN



Output Words



Translated Sentence

#### **1. Input Embedding**

The input sentence is first divided into words or tokens.

For example:

**"Where is my order?"**

Each word is converted into a numerical vector using an embedding layer.

#### **2. Bidirectional RNN Encoder**

A normal RNN processes a sentence in one direction:

Word 1 → Word 2 → Word 3 → Word 4

A Bidirectional RNN processes the sentence in both directions:

**Forward:**

Word 1 → Word 2 → Word 3 → Word 4

**Backward:**

Word 4 → Word 3 → Word 2 → Word 1

The two hidden states are combined.

### **3. Why Bidirectional Processing Helps**

**Consider:**

"The customer cancelled the order because it was delayed."

Understanding the word "it" requires information from surrounding words.

**Bidirectional processing allows the network to consider:**

- Previous words.
- Following words.
- Complete sentence context.

Therefore, it can produce a better representation of the input sentence.

### **4. Encoder**

The encoder converts the complete input sequence into meaningful contextual representations.

It learns:

- Word relationships.
- Sentence structure.
- Context.
- Language patterns.

### **5. Decoder**

The decoder generates the translated sentence one token at a time.

**For example:**

**Input:**

"Where is my order?"

**Output:**

Equivalent translated sentence in the target language.

The decoder uses the encoded representation to determine the next appropriate word.

## **6. Training**

### **During training:**

- Source-language sentences are given to the encoder.
- Correct target-language sentences are given to the decoder.
- The predicted words are compared with the actual target words.
- Backpropagation updates the network parameters.

## **7. Attention Mechanism**

For longer sentences, an attention mechanism can be added.

Attention allows the decoder to focus on the most relevant encoder states while generating each word.

### **For example:**

Input sentence → Relevant words → Output word

This helps preserve important contextual information.

### **Advantages:**

1. Understands context from both directions.
2. Better handling of word relationships.
3. Useful for multilingual translation.
4. Handles variable-length sentences.
5. Encoder–Decoder structure separates understanding and generation.
6. Attention can improve long-sentence translation.

### **Conclusion:**

A Bidirectional RNN Encoder–Decoder is suitable for multilingual chatbot translation because the encoder understands the input from both directions, while the decoder generates the translated sequence. Bidirectional processing provides richer contextual information and can improve translation accuracy, especially when combined with attention.

## Q4. BPTT and LSTM for Stock Price Prediction

### Problem:

A financial analytics company wants to forecast stock prices using historical market data. Stock prices form a time series, where previous values can influence future values.

Therefore, an LSTM-based recurrent neural network trained using Backpropagation Through Time (BPTT) is appropriate.

### Proposed Architecture:

Historical Stock Data



Pre-processing / Normalization



Time-Series Sequences



LSTM Layer



Dense Layer



Predicted Stock Price

### Input Data:

The model can use historical features such as:

- Opening price.
- Closing price.
- Highest price.
- Lowest price.
- Trading volume.
- Previous time-step prices.

### For example:

Day 1 → Day 2 → Day 3 → ... → Day N

The model learns relationships between these observations.

## Why LSTM?

Traditional RNNs can suffer from the vanishing-gradient problem when learning long-term dependencies.

LSTM solves this using a memory cell and gates.

### The main components are:

1. Forget Gate
2. Input Gate
3. Cell State
4. Output Gate

### Forget Gate:

Determines which previous information should be discarded.

### Input Gate:

Determines which new information should be stored in the memory.

### Cell State:

Carries important information through time.

### Output Gate:

Determines the information passed to the next hidden state.

Backpropagation Through Time

BPTT trains recurrent networks by unfolding the network across time.

### For example:

Time 1 → Time 2 → Time 3 → Time 4

The prediction error is propagated backward through these time steps.

The weights are then updated to reduce prediction error.

### Training Process:

1. Historical data is collected.
2. Data is cleaned and normalized.
3. Time-series sequences are created.
4. Sequences are provided to the LSTM.

5. LSTM learns temporal patterns.
6. Prediction is generated.
7. Prediction error is calculated.
8. BPTT updates the network weights.
9. Training continues until the error is minimized.

### Why LSTM is Better Than Traditional RNN?

| Feature               | Traditional RNN    | LSTM               |
|-----------------------|--------------------|--------------------|
| Short-term dependency | Good               | Good               |
| Long-term dependency  | Poorer             | Better             |
| Vanishing gradient    | More likely        | Reduced            |
| Memory mechanism      | Basic hidden state | Cell state + gates |
| Complex time series   | Less suitable      | More suitable      |

### Advantages:

- Learns temporal dependencies.
- Retains important historical information.
- Handles sequential data effectively.
- Reduces the vanishing-gradient problem.
- Suitable for long-term patterns.

### Important Limitation:

Stock prices are affected by many unpredictable factors such as market conditions, economic events and news. Therefore, an LSTM prediction should be treated as a model estimate rather than a guaranteed future price.

### Conclusion:

An LSTM trained using BPTT is more suitable than a traditional RNN for stock-price time-series modelling because LSTM can preserve important information over longer time intervals. Its memory cells and gates help overcome the vanishing-gradient problem and allow the network to learn complex temporal relationships.

## **Q5. Integrated Deep Learning Solution for Automatic Video Captioning**

### **Problem:**

A media-streaming company wants to automatically generate captions for uploaded videos to improve accessibility.

### **The proposed system combines:**

- Transfer Learning
- CNN-based visual feature extraction
- LSTM-based Encoder–Decoder
- Sequence generation

### **Overall Architecture:**

Input Video



Frame Extraction



Pre-trained CNN / Transfer Learning



Visual Feature Vectors



LSTM Encoder



Context Representation



LSTM Decoder



Word-by-Word Caption Generation



Complete Caption

## 1. Video Input

The uploaded video is divided into a sequence of frames.

### For example:

Video  $\rightarrow$  Frame 1  $\rightarrow$  Frame 2  $\rightarrow$  Frame 3  $\rightarrow$  ...  $\rightarrow$  Frame N

Only selected frames may be processed to reduce computational requirements.

## 2. Transfer Learning

A pre-trained CNN such as ResNet or EfficientNet can be used to extract visual features.

Instead of training the CNN from scratch, its previously learned visual representations are reused.

The CNN can identify features such as:

- Objects.
- Shapes.
- People.
- Places.
- Actions.
- Scene characteristics.

This is particularly useful because training a complete visual model from scratch requires a large dataset.

## 3. Feature Extraction

Each video frame is passed through the CNN.

The output is converted into a feature vector:

Frame  $\rightarrow$  CNN  $\rightarrow$  Feature Vector

For multiple frames:

$F_1, F_2, F_3, \dots, F_n$

These feature vectors represent the visual information in the video.

## 4. LSTM Encoder

The sequence of visual features is provided to an LSTM encoder.

The LSTM learns the temporal relationship between frames.

### **For example:**

Frame 1: Person enters room

↓

Frame 2: Person approaches table

↓

Frame 3: Person picks up object

The LSTM can learn the sequence of these events.

## **5. LSTM Decoder**

The decoder generates the caption word by word.

For example, the generated sequence may follow:

→ "A" → "person" → "picks" → "up" → "an" → "object" →

At every step, the decoder predicts the next word based on:

- Previously generated words.
- Encoded video features.
- Learned language patterns.

## **6. Attention Mechanism**

An attention mechanism can improve the architecture by allowing the decoder to focus on important video frames.

### **For example:**

"A person enters a room."

The decoder can focus more strongly on frames showing the person entering the room.

For an action such as picking up an object, it can focus on frames containing that action.

This improves caption relevance.

## **7. Training**

### **During training:**

1. Videos and their correct captions are collected.
2. Frames are extracted.
3. CNN extracts visual features.
4. LSTM encoder processes the feature sequence.
5. Decoder predicts caption words.
6. Predicted captions are compared with actual captions.

7. Loss is calculated.
8. Backpropagation updates model parameters.

### **Loss Function:**

A common choice is categorical cross-entropy loss for next-word prediction.

### **The objective is:**

Minimize the difference between predicted and actual caption words.

## **8. Caption Generation**

After training, a new video is supplied.

The system:

Video → Frames → CNN Features → LSTM Encoder → LSTM Decoder → Caption

The decoder starts with a start token and repeatedly predicts the next word until an end token is produced.

### **Role of Each Component:**

| Component         | Role                                           |
|-------------------|------------------------------------------------|
| Video Input       | Provides the visual content                    |
| Frame Extraction  | Converts video into manageable image sequences |
| Pre-trained CNN   | Extracts visual features                       |
| Transfer Learning | Reuses previously learned visual knowledge     |
| LSTM Encoder      | Learns temporal relationships between frames   |
| Attention         | Focuses on important frames/features           |
| LSTM Decoder      | Generates caption words                        |
| Output Layer      | Predicts the next word                         |
| Training Process  | Optimizes the complete model                   |

## **'Why This Integrated Solution Is Effective**

### **1. Transfer Learning**

Reduces training requirements and provides strong visual representations.

### **2. CNN**

Understands what is present in each frame.

### **3. LSTM**

Understands how the visual information changes over time.

### **4. Encoder–Decoder**

Connects visual understanding with natural-language generation.

### **5. Attention**

Helps the decoder focus on the most relevant visual information.

### **6. Automatic Accessibility**

The generated captions can help users understand video content even when audio is unavailable or difficult to hear.

### **Example:**

#### **Suppose a video shows:**

Person → enters room → sits on chair → opens laptop

The system extracts visual features from the frames, learns their temporal sequence, and may generate a caption such as:

"A person enters a room, sits down, and opens a laptop."

### **Overall Conclusion:**

The five proposed deep-learning solutions address different real-world problems:

| <b>Question</b> | <b>Proposed Solution</b>            | <b>Main Benefit</b>                               |
|-----------------|-------------------------------------|---------------------------------------------------|
| Q1              | Transfer Learning + EfficientNet    | Accurate classification with limited medical data |
| Q2              | DenseNet + Pixel-level segmentation | Detailed road-scene understanding                 |

|    |                                                |                                             |
|----|------------------------------------------------|---------------------------------------------|
| Q3 | Bidirectional RNN Encoder–Decoder              | Better contextual translation               |
| Q4 | LSTM + BPTT                                    | Learning long-term time-series dependencies |
| Q5 | Transfer Learning + CNN + LSTM Encoder–Decoder | Automatic video caption generation          |